

Ordonnancer un bloc opératoire

Guide complet du module `planning.py`

Comprendre les classes, les règles, et savoir s'en servir

Au programme

Les 5 classes et leur rôle · la chronologie d'un patient · ressources disjonctives et cumulatives · les 8 règles dures · le balayage · l'utilisation pas à pas · la référence de l'API · les pièges · le passage au solveur

Sommaire

1 Ce que fait ce code

Le périmètre, et la différence entre vérifier et construire

2 Le vocabulaire du bloc

TVO, TROS, TIS, SSPI, vacation, ambulatoire

3 Vue d'ensemble des 5 classes

Qui porte les données, qui porte les règles

4 La chronologie d'un patient

Trois ressources successives, une seule décision

5 La classe Patient

Attributs, propriétés calculées, et le piège des défauts mutables

6 La classe Medecin

Le motif cyclique et son déroulement en vacances

7 La classe Vacation

TVO, marge de sécurité, capacité utile

8 La classe Conflit

Pourquoi on collecte au lieu de lever une exception

9 Les deux familles de ressources

Disjonctif, cumulatif, et l'algorithme de balayage

10 Les 8 règles dures

Ce que chacune vérifie, et pourquoi

11 Modifier sans casser le planning

Le motif tenter / vérifier / annuler

12 Utilisation pas à pas

Huit étapes, du planning vide aux indicateurs

13 Référence de l'API

Toutes les méthodes en un coup d'oeil

14 Les pièges fréquents

Ce sur quoi on se trompe la première fois

15 Vérifier que vous avez compris

Neuf exercices progressifs

16 Et après : le solveur

Correspondance avec CP-SAT

PARTIE 1

Ce que fait ce code

Le module `planning.py` répond à une question précise : **un planning de bloc opératoire donné est-il valide, et que coûte-t-il ?** Il sait dire si une intervention en gêne une autre, combien de lits sont occupés à un instant donné, combien de temps il reste dans une vacation, et à quel point la courbe d'occupation est chaotique.

Il ne cherche pas la meilleure solution. C'est une distinction importante et souvent mal comprise au démarrage d'un projet d'ordonnancement :

	VÉRIFIER (ce module)	CONSTRUIRE (un solveur)
Question posée	ce planning respecte-t-il les règles ?	quel planning respecte les règles et minimise le coût ?
Entrée	un planning complet	des patients, des vacations, des capacités
Sortie	une liste de conflits et des indicateurs	une affectation
Coût	instantané	de quelques secondes à plusieurs minutes
Outil	du Python standard	CP-SAT, un solveur MILP, une métaheuristique

L'ordre est volontaire. On ne peut pas optimiser ce qu'on ne sait pas évaluer. Un vérificateur fiable est le socle de tout le reste : il sert à tester un solveur, à contrôler une solution avant de l'afficher, et à comparer deux stratégies sur les mêmes données.

À retenir

Ce module est un **juge**, pas un **joueur**. La partie 16 montre comment brancher un joueur dessus, et pourquoi les concepts se transposent presque mot pour mot.

PARTIE 2

Le vocabulaire du bloc

Le code emploie les termes du métier. Les connaître évite beaucoup de malentendus avec les soignants, et ce sont ceux que vous trouverez dans les fichiers de données hospitalières.

Terme	Signification	Dans le code
Vacation	Une plage horaire pendant laquelle une salle est ouverte et attribuée à un praticien ou à une spécialité. C'est l'unité de base de la planification du bloc.	classe Vacation
TVO	Temps de Vacation Offert : la durée de la vacation telle qu'elle est mise à disposition. Une vacation de 8 h à 13 h offre 5 h de TVO.	Vacation.tvo
TROS	Temps Réel d'Occupation de Salle : de l'entrée du patient en salle à sa sortie. C'est ce temps qui remplit la vacation , et non la durée d'incision, qui est plus courte de 20 à 40 minutes.	Patient.duree_op
TIS	Temps Inter-Salle : nettoyage, bionettoyage et installation du patient suivant. Incompressible, souvent 15 à 20 minutes.	Planning.tis
SSPI	Salle de Surveillance Post-Interventionnelle, la salle de réveil. Capacité limitée, et fréquemment le vrai goulot d'étranglement du bloc.	Patient.duree_sspi
Ambulatoire	Le patient entre et sort le même jour. Il occupe une <i>place</i> , pas un lit, et la place se libère le soir.	Patient.ambulatoire
UCA	Unité de Chirurgie Ambulatoire. Elle ferme le soir : un ambulatoire programmé trop tard devient une hospitalisation non prévue.	Planning.fermeture_uca
Taux d'occupation	TROS cumulé divisé par le TVO. L'indicateur de remplissage de référence d'un bloc.	charge_vacation()
Déprogrammation	Une intervention annulée ou repoussée faute de place, de lit ou de temps. L'événement que tout le système cherche à éviter.	deprogrammer()

Le piège de la durée

Si vous calibrez vos durées sur l'incision au lieu du temps de salle, vos vacations de quatre heures déborderont systématiquement d'une heure et vous ne comprendrez pas pourquoi. Dans un export hospitalier, prenez toujours *heure de sortie de salle moins heure d'entrée en salle*.

PARTIE 3

Vue d'ensemble des 5 classes

Le module repose sur un principe simple : **les objets portent les données, le Planning porte les règles**. Un patient ne sait pas s'il gêne quelqu'un, parce qu'il ne voit que lui-même. Seul le Planning a la vue d'ensemble, donc c'est lui qui décide.

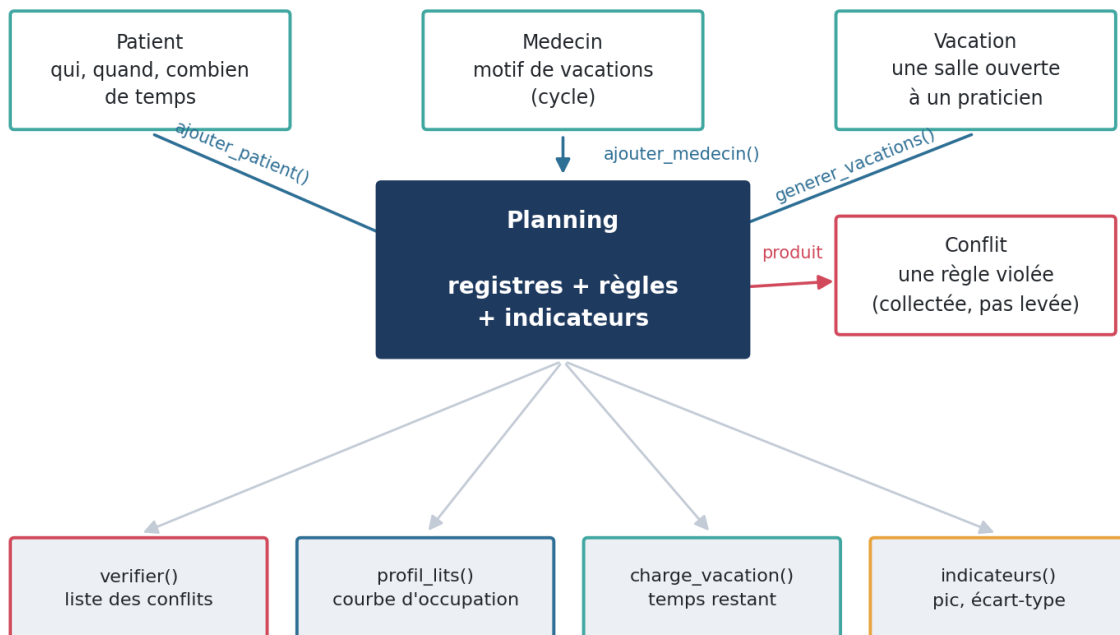


Figure 1 — Les objets alimentent le Planning ; le Planning produit des conflits et des indicateurs.

Classe	Rôle en une phrase	Ce qu'elle ne fait pas
Patient	Qui est opéré, par qui, combien de temps en salle, en réveil et au lit.	Elle ne valide rien.
Medecin	Le motif cyclique des vacations d'un praticien.	Elle ne connaît pas ses patients.
Vacation	Une salle ouverte, à un créneau donné, pour un praticien.	Elle ne connaît pas non plus ses patients : c'est le Planning qui fait le lien.
Conflit	Une règle violée, avec son code et son message.	Ce n'est pas une exception : on la range dans une liste.
Planning	Le registre de tout, les règles, et les indicateurs.	Il ne cherche pas la meilleure solution.

Ce découplage a une conséquence pratique : on peut remplacer n'importe quelle classe de données sans toucher aux règles. Si vos patients viennent d'un fichier Excel plutôt que d'un constructeur Python, seule la construction change.

PARTIE 4

La chronologie d'un patient

Un patient opéré consomme trois ressources différentes, l'une après l'autre. C'est la clé de tout le reste : ce ne sont pas les mêmes ressources, elles n'ont pas les mêmes capacités, et elles ne saturant pas au même moment.

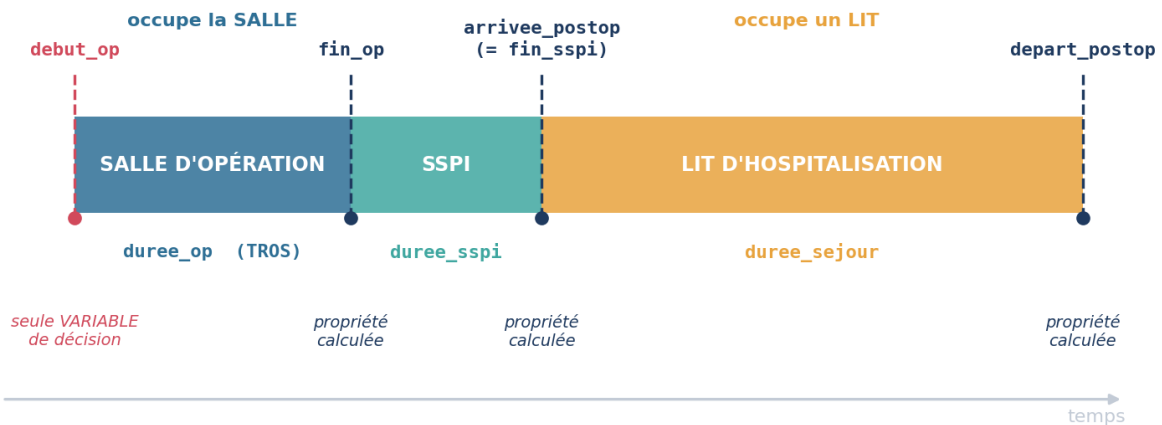


Figure 2 — Une seule variable de décision, trois occupations de ressources qui en découlent.

Un patient occupe **la salle** pendant **duree_op**, puis **la SSPI** pendant **duree_sspi**, puis **un lit** pendant **duree_sejour**. Décalez **debut_op** d'une heure et les trois occupations se décalent ensemble.

La règle d'or : un seul état, tout le reste est calculé

Dans votre première version, quatre attributs étaient stockés côte à côte : le début d'opération, sa durée, l'arrivée en post-opératoire et le départ. Quatre nombres pour décrire une chose qui n'a qu'un seul degré de liberté. Dès que vous déplacez l'intervention, il faut penser à mettre à jour les quatre, et si vous en oubliez un, le planning devient faux silencieusement.

```
# AVANT : quatre attributs à maintenir cohérents à la main
self.debut_op = debut_op
self.arrivee_postop = arrivee_postop      # et si on oublie de le décaler ?
self.depart_postop = depart_postop        # et celui-là ?

# APRÈS : un seul attribut, trois propriétés calculées
self.debut_op = debut_op                  # LA variable de décision

@property
def fin_op(self):
    return self.debut_op + self.duree_op

@property
def arrivee_postop(self):                 # montée au lit = sortie de SSPI
    return self.fin_op + self.duree_sspi

@property
def depart_postop(self):
    return self.arrivee_postop + self.duree_sejour
```

Le code appelant ne change pas : `p.depart_postop` s'écrit toujours pareil. Mais il est maintenant **impossible** d'obtenir un patient incohérent.

Pourquoi ça compte pour la suite

Un solveur raisonne exactement de la même manière : une variable de décision par intervention, et tout le reste exprimé comme une expression de cette variable. En adoptant cette forme dès maintenant, le passage au solveur devient une traduction, pas une réécriture.

PARTIE 5

La classe Patient

Les attributs

Attribut	Type	Rôle
id	int	Identifiant unique, clé dans le registre du Planning.
sexe, age	str, int	Données administratives, utiles plus tard pour prédire la durée.
diag, clim_assoc, gestes	str, list, list	Diagnostic principal (CIM-10), comorbidités, actes (CCAM). Ce sont les variables qui alimenteront un modèle de prédiction de durée.
med_id, bloc_id	int	Chirurgien et salle. Renseignés automatiquement par programmer() à partir de la vacation.
vacation_id	int ou None	Dans quelle vacation le patient est posé. None signifie « en attente ».
debut_op	datetime ou None	La seule variable de décision. None tant que le patient n'est pas programmé.
duree_op	timedelta	Le TROS : entrée en salle jusqu'à la sortie de salle.
duree_sspi	timedelta	Temps passé en salle de réveil.
duree_sejour	timedelta	Occupation du lit après la SSPI.
ambulatorie	bool	Déclenche la règle R8 et compte sur la capacité de places.

Les propriétés calculées

Propriété	Vaut	Sert à
fin_op	debut_op + duree_op	Règles R2 et R4, remplissage de la vacation.
fin_sspi	fin_op + duree_sspi	Règle R7, occupation de la salle de réveil.
arrivee_postop	fin_sspi	Début d'occupation du lit.
depart_postop	arrivee_postop + duree_sejour	Fin d'occupation du lit.
programme	debut_op n'est pas None	Filtrer les patients placés.

Les méthodes

```
p = Patient(id=1, med_id=3,
            duree_op=timedelta(minutes=90),
            duree_sspi=timedelta(minutes=60),
            duree_sejour=timedelta(days=4))

p.temps_post_op()          # -> timedelta(days=4)
p.deplacer_op(timedelta(minutes=30))  # décale TOUT le patient de 30 min
```

`deplacer_op` ne vérifie rien, et c'est voulu. Un patient ne peut pas savoir s'il gêne un autre patient. Passez toujours par `Planning.deplacer()`, qui appelle cette méthode puis contrôle les règles.

Le piège des arguments par défaut mutables

Votre version d'origine contenait un bug très classique et totalement silencieux. En Python, la valeur par défaut d'un paramètre est évaluée **une seule fois**, au moment où la fonction est définie. Une liste par défaut est donc physiquement partagée par toutes les instances.

```
# LE BUG
class Patient:
    def __init__(self, gestes=["AHPA027"]):  # créée UNE SEULE FOIS
        self.gestes = gestes

a = Patient()
b = Patient()
a.gestes.append("NFKA008")
print(b.gestes)      # ['AHPA027', 'NFKA008']  <-- b a changé tout seul
print(a.gestes is b.gestes)  # True : c'est le MÊME objet

# LA CORRECTION
class Patient:
    def __init__(self, gestes=None):
        self.gestes = list(gestes) if gestes else []
```

Pourquoi c'est grave ici

Sur un planning, ce bug ne plante pas : il fausse. Tous vos patients finissent avec la même liste d'actes, donc avec les mêmes durées prédites, et vos statistiques deviennent aberrantes sans qu'aucune erreur ne s'affiche. La même règle vaut pour les dictionnaires et les ensembles.

PARTIE 6

La classe Medecin

Un praticien ne travaille pas au hasard : il a des vacances récurrentes, selon un cycle. Votre encodage `creneau / modulo` est le bon, et il correspond à ce qu'on trouve dans les vraies grilles hospitalières, qui alternent « semaine paire » et « semaine impaire ».

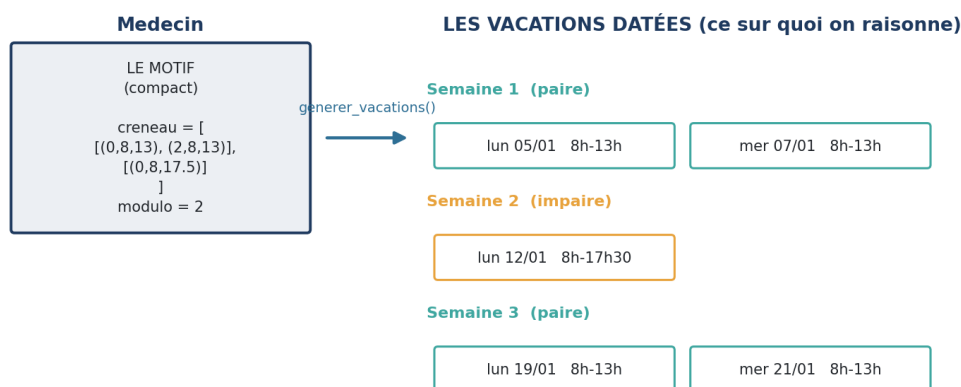
Lire l'encodage

```
ortho = Medecin(
    id=1, nom="JT", specialite="Orthopédie",
    creneau=[
        [(0, 8, 13), (2, 8, 13)], # semaines PAIRES : lundi 8h-13h + mercredi 8h-13h
        [(0, 8, 17.5)],           # semaines IMPAIRES: lundi 8h-17h30
    ],
    modulo=2,                     # le cycle fait 2 semaines
)
```

Élément	Signification
<code>creneau</code>	Liste de semaines. <code>creneau[s]</code> donne les créneaux de la s-ième semaine du cycle.
<code>(jour, h_debut, h_fin)</code>	jour : 0 = lundi, 6 = dimanche. Les heures sont décimales : 17.5 signifie 17h30.
<code>modulo</code>	Longueur du cycle en semaines. La semaine <code>n</code> utilise <code>creneau[n % modulo]</code> .

Dérouler le motif

Le motif est compact, mais on **ne raisonne jamais dessus**. `generer_vacations()` le transforme en objets `Vacation` datés, et toutes les règles travaillent ensuite sur ces objets.



Règle : on ne raisonne jamais sur le motif.
Une fois déroulé, un jour férié ou un congé se retire d'une ligne, sans casser l'arithmétique.

Figure 3 — Du motif compact aux vacances datées, seules manipulables par les règles.

```
lundi = datetime(2026, 1, 5)          # un lundi de référence
vacs = pl.generer_vacations(ortho, lundi, nb_semaines=4,
                             bloc_id=1, debut_id=100)
# -> [<Vacation 100 S1 chir 1 Mon 05/01 08:00-13:00>,
#      <Vacation 101 S1 chir 1 Wed 07/01 08:00-13:00>,
#      <Vacation 102 S1 chir 1 Mon 12/01 08:00-17:30>, ...]
```

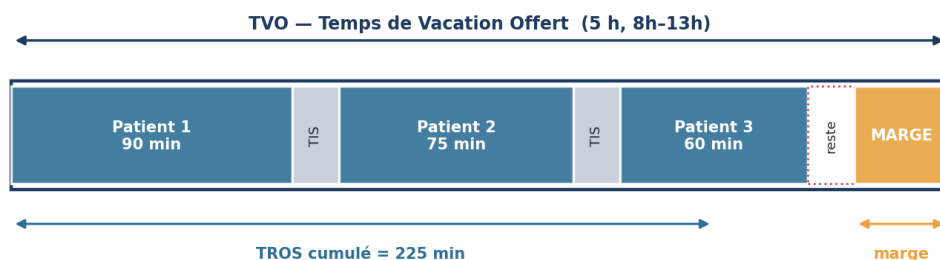
Pourquoi dérouler plutôt que calculer à la volée

Parce que la réalité est pleine d'exceptions : un jour férié, un congrès, un congé, un remplacement, une salle en travaux. Sur une liste d'objets datés, chaque exception est une ligne qu'on retire ou qu'on modifie. Sur un motif, chaque exception devient un cas particulier dans l'arithmétique modulaire, et le code finit par devenir illisible.

PARTIE 7

La classe Vacation

Une vacation, c'est une salle ouverte, à un créneau donné, pour un praticien. C'est le contenant ; les interventions sont le contenu. Toute la question du remplissage du bloc se joue ici.



charge_vacation(v) renvoie : tros = 3 h 45 tis = 30 min utilise = 4 h 15 restant = 15 min taux_occupation = 75 %
capacite_utile = TVO – marge. Le « restant » se calcule sur capacite_utile, jamais sur le TVO brut.

Figure 4 — Une vacation de 5 h ne peut pas recevoir 5 h d'interventions : il faut soustraire les temps inter-salle et la marge.

Attribut ou propriété	Rôle
bloc_id	La salle. Deux vacations de la même salle ne doivent pas se chevaucher.
debut, fin	Les bornes. Une intervention doit tenir entièrement dedans (R4).
med_id	Le praticien à qui la vacation est attribuée.
specialite	Utile si vous affectez par spécialité plutôt que par praticien.
marge	Tampon conservé libre pour absorber les dépassements. 30 min par défaut.
tvo	Temps de Vacation Offert = fin – debut.
capacite_utile	tvo – marge. C'est sur cette valeur, jamais sur le TVO brut, que se calcule le temps restant.

Mesurer le remplissage

```
bilan = pl.charge_vacation(100)

bilan["nb_patients"]      # 3
bilan["tros"]             # 3:45:00  somme des durées de salle
bilan["tis"]              # 0:30:00  2 inter-salles entre 3 patients
bilan["utilise"]          # 4:15:00  tros + tis
bilan["restant"]          # 0:15:00  capacite_utile - utilise
bilan["taux_occupation"]  # 0.75   tros / tvo
bilan["patients"]         # la liste, triée par heure de début
```

Pourquoi une marge

Les durées opératoires sont des variables aléatoires, et leur distribution est asymétrique : beaucoup d'interventions un peu plus courtes que prévu, quelques-unes beaucoup plus longues.

Empiriquement, elles suivent plutôt une loi log-normale qu'une gaussienne.

Un planning construit sur les durées moyennes, sans marge, s'effondre dès le premier dépassement : le patient suivant décale, puis celui d'après, et la vacation finit en heures supplémentaires ou en déprogrammation. La marge est le prix de la robustesse.

Comment dimensionner la marge

C'est un arbitrage économique classique, de la même famille que le problème du vendeur de journaux : trop de marge coûte du temps de salle inutilisé, trop peu coûte des heures supplémentaires et des annulations. Le bon réglage se trouve par simulation, en rejouant un millier de journées avec des durées tirées au sort.

PARTIE 8

La classe Conflit

Quand une règle est violée, le module ne lève pas d'exception. Il crée un objet `Conflit` et le range dans une liste. Ce choix, qui peut surprendre, est déterminant pour la suite du projet.

```
@dataclass
class Conflit:
    regle: str          # "R2", "R6"...
    message: str        # lisible par un humain
    patient_id: int = None
    autre_id: int = None # l'autre patient impliqué, pour R2 et R3
```

Lever une exception	Collecter dans une liste
S'arrête à la première erreur.	Voit toutes les erreurs d'un coup.
On ne sait pas combien il y en a.	On peut compter : c'est un score.
Impossible d'explorer une solution invalide.	On peut accepter temporairement une solution invalide et la réparer.
Difficile à transformer en coût.	Chaque conflit peut recevoir une pénalité et entrer dans une fonction objectif.

Ce dernier point est le vrai motif. Toutes les métaheuristiques (recuit simulé, recherche tabou, recherche à voisinage large) ont besoin de traverser des solutions légèrement invalides pour aller d'un bon optimum local à un meilleur. Avec des exceptions, cette exploration est impossible.

```
conflits = pl.verifier()
print(len(conflits), "violation(s)")
for c in conflits:
    print(c)          # [R3] chirurgien 1 : conflit avec le patient 7

# un score de pénalité, base de toute recherche locale
POIDS = {"R1": 1000, "R2": 100, "R3": 100, "R4": 50,
         "R5": 20, "R6": 30, "R7": 30, "R8": 10}
cout = sum(POIDS[c.regle] for c in conflits)
```

PARTIE 9

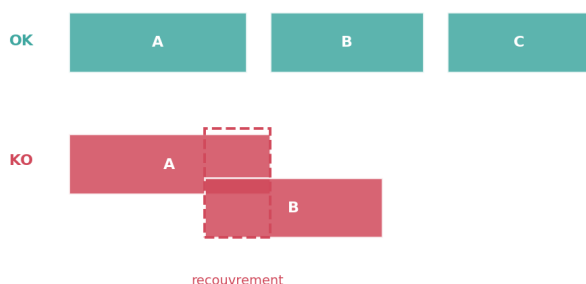
Les deux familles de ressources

C'est le concept central du guide. Vos trois besoins de départ (compter les lits, calculer le temps restant, éviter la concurrence) se ramènent en réalité à **deux types de ressources**, que la littérature de l'ordonnancement traite différemment.

DISJONCTIVE

salle, chirurgien — une seule tâche à la fois

AddNoOverlap / chevauchent()



CUMULATIVE

lits, SSPI — n tâches en parallèle, $n \leq \text{capacité}$

AddCumulative / profil_cumulatif()

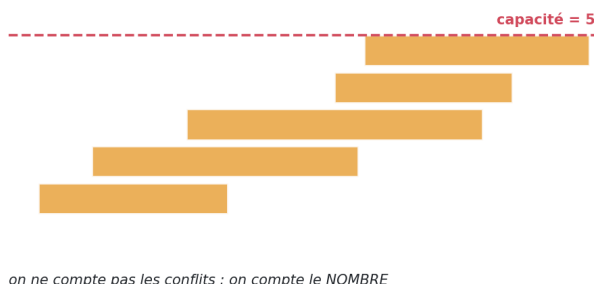


Figure 5 — À gauche, une ressource qu'on ne peut pas partager. À droite, une ressource qu'on partage jusqu'à une limite.

Les ressources disjonctives : salle et chirurgien

Une salle accueille une intervention à la fois. Un chirurgien opère un patient à la fois. La vérification se ramène à un test de recouvrement de deux intervalles.

```
def chevauchent(d1, f1, d2, f2):
    return d1 < f2 and d2 < f1
```

Le piège des intervalles fermés

Le code utilise des intervalles **semi-ouverts**, notés [début, fin[. Un patient qui sort à 10h00 et un autre qui entre à 10h00 ne se chevauchent pas. Avec des comparaisons larges ($\leq$ au lieu de $<$), vous compteriez un conflit à chaque transition, et un lit de trop à chaque changement de patient. C'est l'erreur la plus fréquente sur ce type de code, et la plus difficile à repérer parce qu'elle ne produit qu'un léger décalage systématique.

Le module ajoute au passage deux temps incompressibles souvent oubliés : le temps inter-salle entre deux patients dans la même salle, et le temps de trajet du chirurgien s'il opère dans deux salles différentes.

Les ressources cumulatives : lits et SSPI

Un lit n'est pas une salle. Trente patients peuvent être hospitalisés en même temps si l'établissement a trente lits. La question n'est plus « y a-t-il conflit ? » mais « combien y en a-t-il à cet instant ? ».

La tentation naïve est de parcourir le temps minute par minute et de compter. Sur trois mois, cela fait cent trente mille itérations, répétées à chaque évaluation. C'est inutilisable dans une boucle

d'optimisation.

L'algorithme de balayage

La technique standard s'appelle le balayage, ou ligne de balayage. Elle repose sur une observation simple : **l'occupation ne change qu'aux arrivées et aux départs**. Entre deux de ces instants, elle est constante, donc inutile de la recalculer.

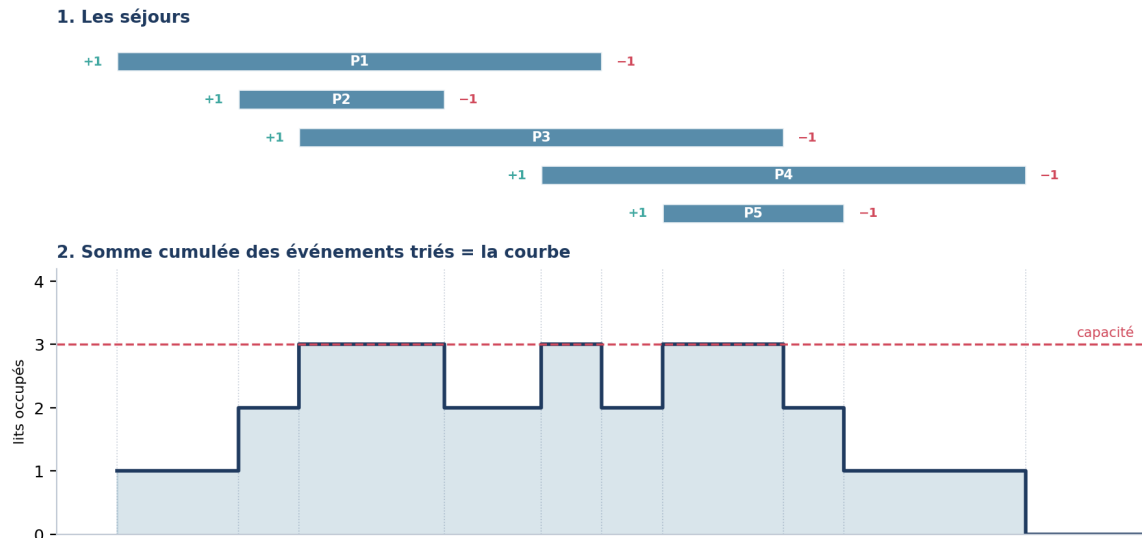


Figure 6 — On note +1 à chaque arrivée et -1 à chaque départ, on trie les instants, on somme. La courbe d'occupation apparaît.

```
def profil_cumulatif(intervalles):
    evenements = defaultdict(int)
    for debut, fin in intervalles:
        evenements[debut] += 1      # une arrivée
        evenements[fin] -= 1       # un départ

    courant, profil = 0, []
    for instant in sorted(evenements):
        courant += evenements[instant]
        profil.append((instant, courant))
    return profil
```

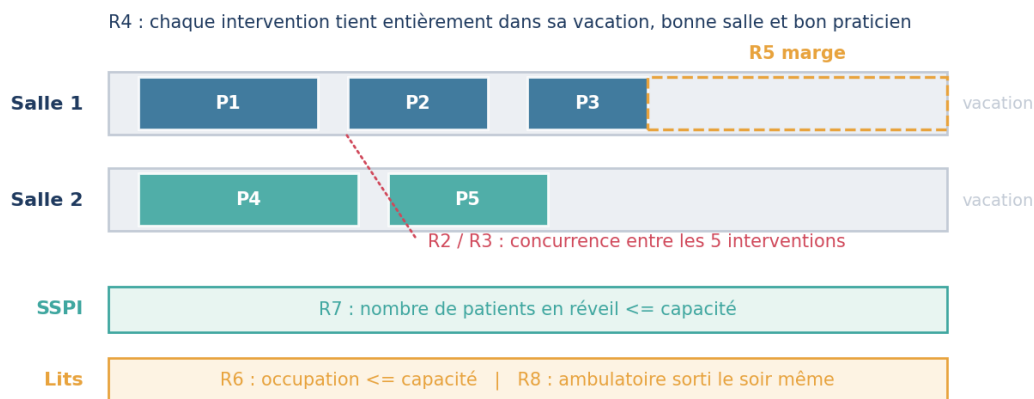
Le coût passe de $O(\text{horizon})$ à $O(n \log n)$, où n est le nombre de patients. Et la fonction est générique : le module l'utilise telle quelle pour les lits d'hospitalisation, les places d'ambulatoire et la salle de réveil.

```
pl.profil_lits()                # [(datetime, nb), ...] la courbe complète
pl.profil_lits(ambulatoire=True) # la même chose pour les places
pl.profil_sspi()               # et pour la salle de réveil
pl.pic_lits()                  # (instant du pic, valeur du pic)
pl.occupation_lits(instant)    # la valeur à un instant précis
pl.depassements(profil, 42)    # [(début, fin, niveau), ...] au-dessus de 42
```

PARTIE 10

Les 8 règles dures

Une règle dure **interdit**. Si elle est violée, la modification est annulée. Voici les huit que le module vérifie, dans l'ordre où `conflits_patient()` les teste.



R1 cohérence · R2 salle · R3 chirurgien · R4 vacation · R5 marge · R6 lits · R7 SSPI · R8 ambulatoire

Dures : elles **INTERDISENT**. Une violation annule la modification.

Souples (S1 à S6) : elles **PÉNALISENT**. Jamais en interdiction, sinon le problème devient infaisable.

Figure 7 — Où s'applique chaque règle sur une journée de bloc.

Code	Ce qu'elle vérifie	Type	Exemple de déclenchement
R1	Cohérence interne : durée opératoire strictement positive, durée de séjour non négative.	Intégrité	Une durée saisie à zéro dans un import de données.
R2	Deux interventions de la même salle ne se recouvrent pas, temps inter-salle inclus.	Disjonctive	On avance un patient de 30 min et il mord sur le précédent.
R3	Un praticien n'opère pas deux patients en même temps, temps de trajet inclus.	Disjonctive	Un chirurgien se voit attribuer deux salles en parallèle.
R4	L'intervention tient entièrement dans une vacation ouverte, de la bonne salle et du bon praticien.	Appartenance	Une intervention de 2 h posée à 12h dans une vacation qui ferme à 13h.
R5	Le cumul TROS + TIS ne dépasse pas la capacité utile (TVO moins la marge).	Capacité	Un quatrième patient ajouté dans une vacation déjà pleine à 90 %.
R6	L'occupation des lits reste sous la capacité, à tout instant.	Cumulative	Trop d'interventions lourdes programmées le même jour.
R7	L'occupation de la SSPI reste sous sa capacité.	Cumulative	Trois sorties de salle simultanées avec deux postes de réveil.
R8	Un ambulatoire entre et sort le même jour, avant la fermeture de l'UCA.	Métier	Un ambulatoire programmé à 16h avec 5 h de séjour.

Les règles souples

Six autres règles sont documentées dans le code mais volontairement **non implémentées comme des interdictions** :

Code	Règle	Pourquoi elle est souple
S1	Lisser l'occupation des lits.	C'est un objectif, pas une contrainte : il n'y a pas de seuil au-delà duquel c'est interdit.
S2	Éviter les séjours qui traversent le week-end sans raison médicale.	Parfois inévitable.
S3	Placer les ambulatoires en début de vacation.	Pour qu'ils puissent sortir le soir même, mais ce n'est pas toujours possible.
S4	Placer les patients septiques en fin de vacation.	Pratique d'hygiène, pas une obligation absolue.
S5	Opérer tôt les enfants et les diabétiques.	À cause du jeûne préopératoire. Fortement souhaitable, pas interdit sinon.
S6	Regrouper les actes similaires dans une même vacation.	Gain de temps par effet d'apprentissage et matériel commun.

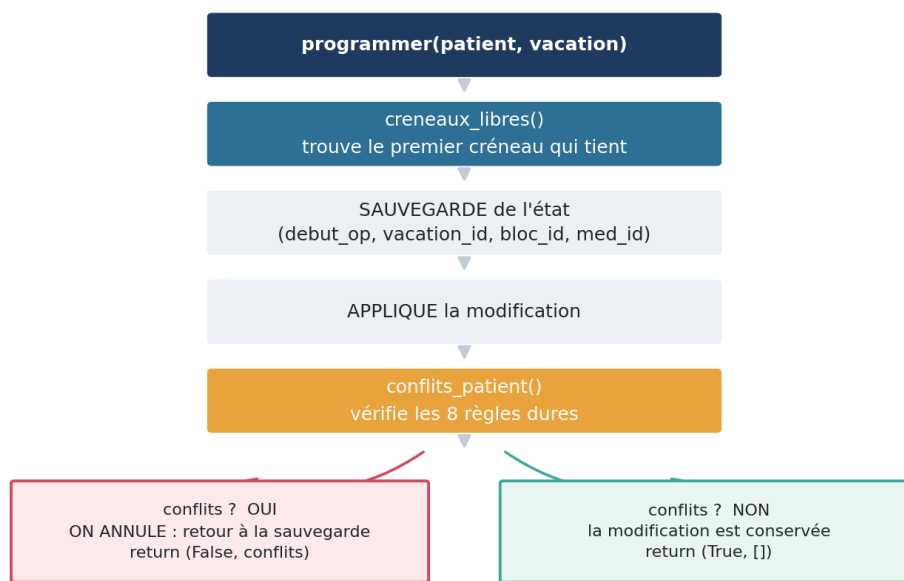
La règle la plus importante de cette page

Ne transformez **jamais** une règle souple en règle dure. Si vous interdisez les séjours qui traversent le week-end, vous rendrez le problème infaisable dès la première semaine chargée, et le solveur vous répondra qu'il n'existe aucune solution, sans vous dire laquelle de vos exigences est de trop. Les préférences se mettent dans la fonction objectif, sous forme de pénalités.

PARTIE II

Modifier sans casser le planning

Trois méthodes modifient le planning : `programmer()`, `deplacer()` et `deprogrammer()`. Les deux premières suivent exactement le même schéma, celui qui garantit qu'on ne laisse jamais le planning dans un état invalide.



*Le planning n'est JAMAIS laissé dans un état invalide.
deplacer() suit exactement le même schéma.*

Figure 8 — Le motif tenter / vérifier / annuler.

```

def deplacer(self, patient_id, delta, forcer=False):
    p = self.patients[patient_id]
    sauvegarde = p.debut_op                # 1. on retient l'état

    p.deplacer_op(delta)                   # 2. on applique
    conflits = self.conflits_patient(p)    # 3. on vérifie

    if conflits and not forcer:
        p.debut_op = sauvegarde            # 4. on annule
        return False, conflits
    return True, conflits

```

Méthode	Signature	Retour
programmer	<code>programmer(patient_id, vacation_id, debut=None, forcer=False)</code>	(True, []) si placé, sinon (False, liste de conflits)
deplacer	<code>deplacer(patient_id, delta, forcer=False)</code>	idem
deprogrammer	<code>deprogrammer(patient_id)</code>	rien : remet debut_op et vacation_id à None

Si `debut` n'est pas fourni, `programmer()` appelle `creneaux_libres()` et prend le premier créneau qui tient. C'est une stratégie dite *first-fit*, empruntée aux algorithmes de rangement en boîtes.

Les points d'ancrage

Pour trouver où poser une intervention, inutile de tester toutes les minutes de la vacation. Une solution optimale existe toujours en collant une intervention soit au début de la vacation, soit juste après une autre. Ces quelques instants s'appellent les points d'ancrage.

```
libres = pl.creneaux_libres(vacation_id=100,
                           duree=timedelta(minutes=75))
# -> [datetime(2026,1,5,8,0), datetime(2026,1,5,11,15)]
#     seulement 2 candidats au lieu de 300 minutes à tester
```

Une nuance pour plus tard

Le motif appliquer-puis-annuler est simple et sûr, mais il coûte une vérification complète à chaque essai. Un solveur teste des millions de mouvements par seconde : il évalue donc l'effet d'un déplacement *avant* de l'appliquer, en ne recalculant que ce qui change. On appelle cela l'évaluation incrémentale, et c'est la première optimisation à faire quand votre recherche locale devient trop lente.

PARTIE 12**Utilisation pas à pas**

Huit étapes, du planning vide jusqu'aux indicateurs. Chaque bloc de code se copie tel quel.

Étape 1 • Importer

Le module n'a besoin que de la bibliothèque standard.

```
from datetime import datetime, timedelta
from planning import Planning, Patient, Medecin, Vacation
```

Étape 2 • Créer le planning et déclarer les capacités

Tout se règle ici : capacités, temps inter-salle, heure de fermeture de l'ambulatorio.

```
pl = Planning(
    capacite_lits=42,                # lits d'hospitalisation complète
    capacite_ambu=12,               # places d'ambulatorio
    capacite_sspi=6,                # postes de réveil
    tis=timedelta(minutes=15),      # temps inter-salle
    trajet_chirurgien=timedelta(minutes=10),
    fermeture_uca=20,               # l'ambulatorio ferme à 20h
)
```

Étape 3 • Déclarer les praticiens

Un praticien, sa spécialité, et son motif de vacations.

```
ortho = Medecin(id=1, nom="JT", specialite="Orthopédie",
    creneau=[[(0, 8, 13), (2, 8, 13)], # semaines paires
              [(0, 8, 17.5)]],         # semaines impaires
    modulo=2)
digest = Medecin(id=2, nom="SR", specialite="Digestif",
    creneau=[[(1, 8, 13), (3, 13.5, 17.5)]], modulo=1)

pl.ajouter_medecin(ortho)
pl.ajouter_medecin(digest)
```

Étape 4 • Générer les vacations

On déroule les motifs sur un horizon, en attribuant une salle à chacun. On peut aussi ajouter une vacation à la main, par exemple pour une ouverture exceptionnelle.

```
lundi = datetime(2026, 1, 5)
pl.generer_vacations(ortho, lundi, nb_semaines=4, bloc_id=1, debut_id=100)
pl.generer_vacations(digest, lundi, nb_semaines=4, bloc_id=2, debut_id=200)

# une vacation exceptionnelle, ajoutée à la main
pl.ajouter_vacation(Vacation(id=999, bloc_id=3, med_id=1,
    debut=datetime(2026, 1, 9, 8, 0),
    fin=datetime(2026, 1, 9, 13, 0),
    marge=timedelta(minutes=45)))
```

Étape 5 • Créer les patients

Les durées viendront plus tard d'un modèle prédictif ; pour l'instant on les saisit. Attention à bien utiliser le temps de salle, pas l'incision.

```
pl.ajouter_patient(Patient(
    id=1, med_id=1, age=71, sexe="F",
    diag="M17.1", gestes=["NFKA008"],
    duree_op=timedelta(minutes=90),          # TR05
    duree_sspi=timedelta(minutes=60),
    duree_sejour=timedelta(days=4),
))
pl.ajouter_patient(Patient(
    id=2, med_id=1, duree_op=timedelta(minutes=60),
    duree_sspi=timedelta(minutes=45),
    duree_sejour=timedelta(hours=5), ambulatorie=True,
))
```

Étape 6 • Programmer

Sans date, le module choisit le premier créneau disponible. On peut aussi imposer une heure précise.

```
ok, conflits = pl.programmer(1, vacation_id=100)
if not ok:
    for c in conflits:
        print("refusé :", c)

# imposer une heure
ok, conflits = pl.programmer(2, 100, debut=datetime(2026, 1, 5, 10, 0))

# forcer malgré les conflits (utile pour étudier une situation dégradée)
ok, conflits = pl.programmer(2, 100, forcer=True)
```

Étape 7 • Déplacer, vérifier

Le déplacement est refusé s'il crée un conflit. La vérification globale passe tout le planning en revue.

```
ok, conflits = pl.deplacer(1, timedelta(minutes=-30))
print("déplacement", "accepté" if ok else "refusé")

for c in pl.verifier():
    print(c)          # [R2] salle 1 : conflit avec le patient 3 (08:00-09:30)

print(pl.afficher_journee(datetime(2026, 1, 5)))
```

Étape 8 • Mesurer

C'est ici que le planning devient un objet d'étude, et non plus seulement un agenda.

```
for cle, valeur in pl.indicateurs().items():
    print(f"{cle:<24} {valeur}")

# patients_programmes      3
# patients_en_attente      2
# taux_occupation_moyen    0.19
# lits_moyen              1.8
# lits_ecart_type          1.166
# lits_pic                 3
# lits_creux               0
# depassements_lits        0
# conflits                 0
```

La sortie de `afficher_journee()`

Un diagramme de Gantt textuel, pour vérifier d'un coup d'oeil sans dépendance graphique :

```
=== Monday 05/01/2026 ===
Salle 1 08:00-13:00 chir 1 | TV0 5:00:00 | occup. 75% | reste 0:15:00
  08:00-09:30 patient 1   M17.1   4j
  09:45-11:00 patient 2   M17.1   3j
  11:15-12:15 patient 3   M17.1   AMBU
Salle 2 08:00-13:00 chir 2 | TV0 5:00:00 | occup. 0% | reste 4:30:00
Lits occupés à 12h : 2/6
```

Ce que mesurent vraiment les indicateurs

Le taux d'occupation ne suffit pas. Un bloc peut afficher 85 % d'occupation et produire une courbe de lits catastrophique. L'indicateur qui traduit l'objectif de lissage, c'est **l'écart-type de l'occupation des lits**.

La figure ci-dessous compare deux stratégies sur les mêmes 90 patients, les mêmes vacances et la même capacité. La première, en rouge, place chaque patient au plus tôt : c'est le fonctionnement spontané d'un bloc. La seconde, en vert, choisit parmi les vacances possibles celle qui minimise le pic de lits.

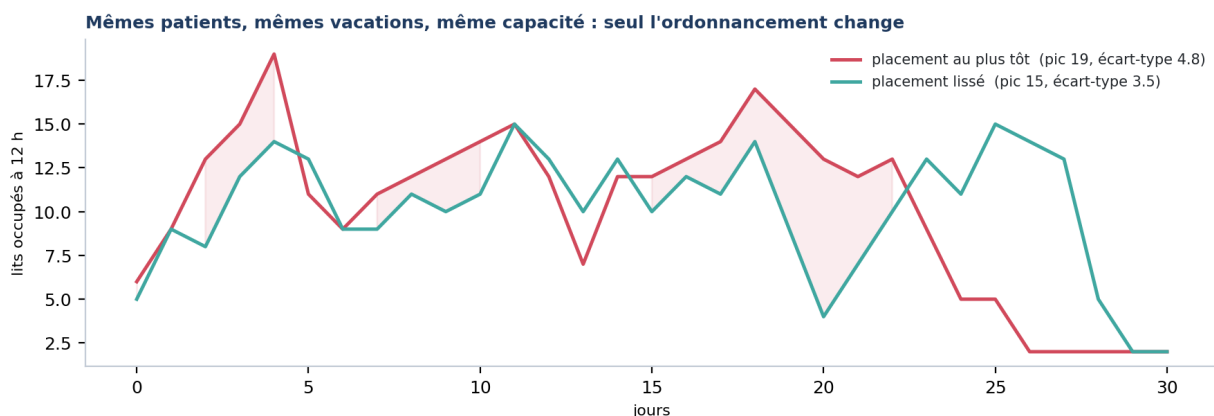


Figure 9 — Sortie réelle du module. Le pic passe de 19 à 15 lits et l'écart-type de 4,8 à 3,5, sans ouvrir une seule vacation supplémentaire.

Quatre lits de pic en moins, c'est une unité d'hospitalisation qu'on n'a pas besoin d'ouvrir. La seule chose qui a changé est l'ordre dans lequel les patients ont été placés.

PARTIE 13

Référence de l'API

Construire

Méthode	Effet
<code>Planning(capacite_lits, capacite_ambu, capacite_sspi, tis, trajet_chirurgien, fermeture_uca)</code>	Crée le planning et fixe les capacités.
<code>ajouter_medecin(m)</code>	Enregistre un praticien.
<code>ajouter_vacation(v)</code>	Enregistre une vacation.
<code>ajouter_patient(p)</code>	Enregistre un patient, non programmé.
<code>generer_vacations(med, lundi_ref, nb_semaines, bloc_id, debut_id)</code>	Déroule le motif d'un praticien en vacances datées. Renvoie la liste créée.

Lire

Méthode	Renvoie
<code>programmes()</code>	La liste des patients ayant une date.
<code>patients_de(vacation_id)</code>	Les patients d'une vacation, triés par heure.
<code>occupation_lits(instant, ambulatorio=False)</code>	Un entier : le nombre de lits occupés.
<code>profil_lits(ambulatorio=False)</code>	La courbe complète : [(instant, nombre), ...].
<code>profil_sspi()</code>	La même chose pour la salle de réveil.
<code>pic_lits(ambulatorio=False)</code>	(instant du maximum, valeur du maximum).
<code>depassements(profil, capacite)</code>	[(début, fin, niveau)] des périodes au-dessus de la capacité.
<code>charge_vacation(vacation_id)</code>	Un dictionnaire : tros, tis, utilise, restant, taux_occupation, patients.
<code>creneaux_libres(vacation_id, duree)</code>	Les heures de début possibles.
<code>afficher_journee(jour)</code>	Une chaîne de caractères, le Gantt textuel.

Vérifier et modifier

Méthode	Renvoie
<code>conflits_patient(p)</code>	Les conflits concernant ce patient.
<code>verifier()</code>	Tous les conflits du planning, dédoublonnés.
<code>programmer(patient_id, vacation_id, debut=None, forcer=False)</code>	(succès, conflits).
<code>deplacer(patient_id, delta, forcer=False)</code>	(succès, conflits).

Méthode	Renvoie
<code>deprogrammer(patient_id)</code>	Rien. Le patient redevient en attente.
<code>indicateurs(heure_mesure=12)</code>	Le tableau de bord complet.

Fonctions libres

Fonction	Renvoie
<code>chevauchent(d1, f1, d2, f2)</code>	True si <code>[d1,f1[</code> et <code>[d2,f2[</code> se recouvrent.
<code>inclus(d1, f1, d2, f2)</code>	True si <code>[d1,f1[</code> est contenu dans <code>[d2,f2[</code> .
<code>profil_cumulatif(intervalles)</code>	Le profil par balayage : <code>[(instant, nombre), ...]</code> .

PARTIE 14

Les pièges fréquents

#	Le piège	La conséquence
1	Confondre durée d'incision et temps de salle.	Vos vacations débordront de 20 à 40 minutes par patient. Prenez toujours sortie de salle moins entrée en salle.
2	Utiliser des comparaisons larges dans le test de chevauchement.	Chaque transition devient un faux conflit et un lit fantôme. Les intervalles sont semi-ouverts.
3	Oublier le temps inter-salle.	Une vacation paraît remplie à 100 % alors qu'elle déborde déjà en pratique.
4	Calculer le temps restant sur le TVO brut.	Vous mangez la marge sans vous en apercevoir. Utilisez toujours <code>capacite_utile</code> .
5	Stocker l'état dérivé.	Le premier déplacement rend le patient incohérent, sans erreur visible. Une seule variable de décision, le reste en propriétés.
6	Mettre une liste ou un dictionnaire en argument par défaut.	Toutes vos instances partagent le même objet. Utilisez <code>None</code> .
7	Transformer une préférence en interdiction.	Le problème devient infaisable et le solveur ne vous dira pas laquelle de vos exigences est de trop.
8	Compter les lits minute par minute.	Cent trente mille itérations par évaluation. Passez par le balayage.
9	Raisonner sur le motif cyclique au lieu des vacations datées.	Le premier jour férié fait exploser le code en cas particuliers.
10	Optimiser le bloc seul.	Vous remplissez parfaitement les salles et vous saturez les lits trois jours plus tard. Les deux ressources se règlent ensemble.
11	Planifier sur les durées moyennes.	Un planning sans marge s'effondre au premier dépassement. Les durées opératoires ont une distribution asymétrique.

PARTIE 15**Vérifier que vous avez compris**

Neuf exercices, du plus simple au plus ouvert. Les premiers se résolvent en quelques lignes avec l'API existante ; les derniers demandent d'étendre le module.

#	Niveau	Énoncé
1	Facile	Écrivez une fonction qui renvoie, pour une semaine donnée, la vacation la moins bien remplie du bloc.
2	Facile	Affichez la liste des jours où l'occupation des lits dépasse 80 % de la capacité.
3	Facile	Comptez combien de patients en attente pourraient tenir dans une vacation donnée, sans les programmer.
4	Moyen	Ajoutez une contrainte de spécialité : une salle équipée pour l'orthopédie ne doit pas recevoir de digestif. Où placez-vous la vérification, et pourquoi ?
5	Moyen	Implémentez la règle souple S3 sous forme de pénalité : un ambulatoire programmé après 14h coûte 10 points. Écrivez la fonction de coût du planning.
6	Moyen	Écrivez une stratégie de remplissage qui, pour chaque patient, choisit la vacation minimisant le pic de lits. Comparez l'écart-type obtenu avec celui du placement au plus tôt.
7	Difficile	Implémentez l'évaluation incrémentale : une fonction qui calcule le coût d'un déplacement sans modifier le planning ni recalculer tout le profil.
8	Difficile	Ajoutez la variabilité : tirez les durées opératoires selon une loi log-normale calibrée sur les données historiques, rejouez mille fois une même journée, et mesurez la probabilité de dépassement de vacation. Faites varier la marge et tracez la courbe.
9	Ouvert	Le module suppose les urgences absentes. Comment les intégreriez-vous ? Réservation de plages dédiées, marge accrue, ou replanification à chaque arrivée ? Comparez les trois sur simulation.

PARTIE 16

Et après : le solveur

Le second fichier, `ordonnancement_cpsat.py`, résout le même problème avec CP-SAT, le solveur de programmation par contraintes d'OR-Tools. Le point remarquable est que les concepts se transposent presque terme à terme.

Dans <code>planning.py</code>	Dans CP-SAT	Ce que ça exprime
<code>debut_op</code>	une variable entière	Les minutes écoulées depuis une origine.
<code>chevauchent()</code> sur une salle	<code>AddNoOverlap</code>	Ressource disjonctive.
<code>chevauchent()</code> sur un chirurgien	<code>AddNoOverlap</code>	Ressource disjonctive.
<code>profil_lits()</code> comparé à la capacité	<code>AddCumulative</code>	Ressource cumulative.
<code>vacation_id</code>	<code>NewOptionalIntervalVar</code> et son littéral de présence	« ce patient est-il opéré dans cette vacation ? »
<code>charge_vacation()</code>	les bornes de l'intervalle optionnel	R4 et R5.
les pénalités des règles souples	<code>Minimize(...)</code>	La fonction objectif.

Deux différences de fond méritent d'être soulignées.

Le temps devient entier. Un solveur ne manipule pas d'objets `datetime`. On choisit une origine et une granularité, typiquement la minute, et tout devient arithmétique sur des entiers. La conversion se fait à l'entrée et à la sortie.

On ne programme plus l'algorithme, on décrit le problème. Passer de « minimiser le pic de lits » à « lisser l'occupation jour par jour » revient à changer la ligne de l'objectif, pas à réécrire une heuristique.

```
# le pic de lits devient une VARIABLE, et la capacité de la contrainte
pic = model.NewIntVar(0, capacite_lits, "pic_lits")
model.AddCumulative(intervalles_lits, [1] * n, pic)
model.Minimize(1000 * pic + sum(debuts))
```

Une progression raisonnable

#	Méthode	Ce que vous y gagnez
1	Glouton avec règle de priorité : trier par durée décroissante et insérer au premier créneau valide.	Une première solution en quelques millisecondes, et une référence à battre.
2	Recherche locale, puis recherche à voisinage large : détruire 10 % du planning et le reconstruire.	La méthode qui domine en pratique sur ce problème, et elle se code directement sur la classe <code>Planning</code> .
3	CP-SAT.	L'optimalité prouvée sur des instances de taille moyenne, et une modélisation déclarative.
4	Simulation de Monte-Carlo par-dessus.	La robustesse : un planning évalué sur mille réalisations des durées, pas sur une seule.

Le niveau qu'on oublie

L'essentiel du gain sur l'occupation des lits ne se joue pas dans l'affectation patient par patient, mais un cran au-dessus : dans la construction de la grille de vacations elle-même. La littérature appelle cela le *Master Surgical Schedule*, et le lissage des lits en aval est un critère de conception connu de cette grille. Un excellent ordonnancement opérationnel sur une mauvaise grille restera limité.

Les deux fichiers

`planning.py` — structure de données, règles et indicateurs. Bibliothèque standard uniquement.

`ordonnancement_cpsat.py` — le même problème confié à CP-SAT. Nécessite `pip install ortools`.